Take-Home Code Challenge

Automation Quality Assurance Engineer

Mid-Level Position • JavaScript / TypeScript

API Test Automation + UI Test Automation

Estimated completion time: 4–6 hours

1. Challenge Overview

This take-home assignment evaluates your practical skills in test automation using JavaScript or TypeScript. You will build a small but complete test automation suite that covers both API and UI testing against a publicly available demo application.

🎯 What We're Looking For

We care about quality over quantity. A smaller number of well-written, well-structured tests is far more valuable than a large number of hastily assembled scripts.

We're evaluating your engineering approach: how you organize code, handle edge cases, write assertions, and document your decisions.

Technology Stack

• Language: JavaScript or TypeScript (your choice)

• UI Automation: Playwright (preferred), Cypress, or Selenium WebDriver

• API Automation: Playwright API testing, Axios, SuperTest, or any HTTP client

• Test Runner: Playwright Test, Jest, Mocha, or equivalent

• Assertions: Built-in framework assertions or Chai

• Reporting: Any HTML or JSON reporter (Allure, Playwright HTML, mochawesome, etc.)

arget Application

You will test against the following publicly available demo applications. These are free, open services designed for testing practice — no account or API key required.

Component

URL

Purpose

API Under Test

https://reqres.in/

RESTful API with users, registration, and resource endpoints

UI Under Test

https://opensource-demo.orangehrmlive.com

Orange HRM Demo — HR management system with login, employee directory, and more

UI Credentials

Username: Admin | Password: admin123

Pre-configured demo credentials

⚠️ **Important Note**

If any of the above URLs become unavailable during your work, you may substitute an equivalent public demo application. Just document the replacement and your reasoning in the README.

2. Part A — API Test Automation

Build an automated API test suite for the ReqRes API (https://reqres.in/). Your suite should demonstrate your ability to validate RESTful services comprehensively.

Required Test Scenarios

Implement automated tests for the following scenarios. Each test must include meaningful assertions.

A1. User CRUD Operations

1. GET /api/users?page=2 — Verify response status, pagination structure, and that the returned user list is not empty.

2. GET /api/users/{id} — Verify a single user's data structure and validate specific fields (id, email, first_name, last_name).

3. POST /api/users — Create a new user and verify the response includes the submitted name and job, plus a generated id and timestamp.

4. PUT /api/users/{id} — Update an existing user and validate the response reflects the changes.

5. DELETE /api/users/{id} — Verify the response returns a 204 status code.

A2. Authentication & Negative Testing

6. POST /api/register — Successful registration with a valid payload; verify the returned token.

7. POST /api/register — Failed registration with a missing password; verify appropriate error response.

8. POST /api/login — Successful login; verify token is returned.

9. POST /api/login — Failed login with missing credentials; verify error message.

A3. Advanced Scenarios

10. GET /api/users/{id} with a non-existent ID — Verify 404 response and empty body.

11. GET /api/users?delay=3 — Verify the response is received within a reasonable timeout and that data is still valid.

12. Response schema validation — Validate the JSON response structure of at least one endpoint against a defined schema (using a library like ajv or joi).

✅ **Technical Requirements for Part A**

• Use a base URL configuration (not hardcoded in each test)

• Implement a reusable API client or helper module for HTTP calls

• Include at least one data-driven test (same test logic, multiple data sets)

• Implement proper response schema validation on at least one endpoint

• All tests must be independently runnable (no test order dependency)

3. Part B — UI Test Automation

Build an automated UI test suite for the OrangeHRM Demo application. Your suite should demonstrate your ability to interact with web elements, handle waits, and design maintainable test code.

Required Test Scenarios

B1. Login Functionality

1. Successful login — Enter valid credentials, verify the user is redirected to the dashboard, and the dashboard page loads completely.

2. Failed login (invalid password) — Verify the appropriate error message is displayed.

3. Failed login (empty fields) — Verify that required-field validation messages appear.

4. Logout — Log in, then log out, and verify the user is returned to the login page.

B2. Employee Directory (PIM Module)

5. Navigate to PIM — After login, navigate to the PIM module and verify the employee list is displayed.

6. Search for an employee — Use the search functionality to look up an employee by name. Verify that results are shown or an appropriate message appears.

7. Add a new employee — Navigate to Add Employee, fill in the required fields, save, and verify the employee was created successfully.

B3. Dashboard Validation

8. Dashboard widgets — After login, verify that key dashboard elements/widgets are visible and rendered.

9. Sidebar navigation — Verify that clicking sidebar menu items navigates to the correct pages.

✅ **Technical Requirements for Part B**

• Implement the Page Object Model (POM) pattern with at least 3 page objects (Login, Dashboard, PIM)

• Use explicit waits (never hardcoded sleeps/delays in final code)

• Include at least one custom reusable utility (e.g., login helper, screenshot on failure, data generator)

• Tests must be runnable in headless mode

• Include at least one test that captures a screenshot on failure (automatic or manual)

4. Project Structure & Deliverables

Organize your project with a clear, professional folder structure. Below is a suggested layout — you may adapt it to your chosen framework, but the general separation of concerns should be maintained.

```
project-root/
 ├── README.md
 ├── package.json
 ├── tsconfig.json (if using TypeScript)
 ├── playwright.config.ts (or equivalent framework config)
 ├── .env.example (environment variables template)
 │
 ├── src/
 │   ├── api/
```

```
│ │ ├── clients/ (API client/helpers)
│ │ ├── schemas/ (JSON schemas for validation)
│ │ └── tests/ (API test files)
│ │
│ ├── ui/
│ │ ├── pages/ (Page Object classes)
│ │ ├── tests/ (UI test files)
│ │ └── fixtures/ (test data files)
│ │
│ └── utils/ (shared utilities, helpers)
│
├── reports/ (generated test reports)
└── .github/ (optional: CI workflow file)
```

Submission Checklist

Deliverable

Details

README.md

Project description, setup instructions (install, configure, run), design decisions, known issues, and any assumptions made.

Working test suite

All tests pass when run with a single command (e.g., npm test or npx playwright test).

Environment config

A .env.example file with all required variables documented (no hardcoded secrets).

Test report

Include a generated report or instructions on how to generate one.

Git history

Submit as a Git repository (GitHub, GitLab, or zip). We value clean, incremental commits over a single large commit.

No node_modules

Ensure node_modules is gitignored; the project should install cleanly via npm install.

5. Evaluation Rubric

Your submission will be evaluated across five dimensions. Each is scored from 1 (Insufficient) to 4 (Advanced). A total score of 15+ out of 20 is considered a strong pass.

Dimension (Weight)

What "Good" Looks Like (3–4)

Red Flags (1–2)

Code Quality & Readability

(25%)

Clean, consistent code style. Meaningful variable/function names. Proper use of TypeScript types (if applicable). No dead code or commented-out blocks.

Messy formatting, unclear naming, inconsistent patterns, excessive comments.

Framework Architecture

(25%)

Clear separation of concerns (POM, API clients, utilities). Reusable components. Configuration externalized. Easy to add new tests.

Tests with inline locators, duplicated code, hardcoded values throughout, no clear structure.

Test Design & Coverage

(20%)

Tests cover happy paths, negative scenarios, edge cases, and boundary conditions. Data-driven approach where appropriate. Independent test execution.

Only happy-path tests. Tests depend on each other. No negative/edge case coverage.

Assertions & Validation

(15%)

Specific, meaningful assertions. Schema validation. Validates both status codes and response bodies. Clear failure messages.

Only status code checks. Generic or missing assertions. No schema validation.

Documentation & Delivery

(15%)

Comprehensive README with setup steps. Clean git history. Working out-of-the-box after npm install. Report generation included.

Missing README. Single mega-commit. Broken dependencies. No instructions.

Bonus Points (Optional, Not Required)

These are not required but will be noted positively:

• CI/CD Integration — Include a GitHub Actions or GitLab CI workflow that runs your tests automatically.

• Docker Support — Provide a Dockerfile or docker-compose.yml to run tests in a containerized environment.

• Custom Reporter — Implement or configure a reporter that generates a polished HTML or Allure report.

• Visual Regression — Include at least one visual comparison test (e.g., using Playwright's toHaveScreenshot).

• Linting & Formatting — ESLint and Prettier configured with pre-commit hooks.

• Retry Logic — Demonstrate proper retry configuration for flaky-prone scenarios.